Auditing an AI-Assisted Matrix Factorization Recommender on MovieLens 100k: a Regularizer that Erases the Model, an Unrestored Checkpoint, and an RMSE Gain that Loses to Popularity at Top-10

Student: Anastasia Zaporozhets **Team:** Individual
Email: anasalekzaporozhets@edu.hse.ru **Date:** 2026-10-13
Assignment: A05 - Matrix Factorization with TensorFlow.js (Week 5)

Abstract - The Week 5 starter asks for six TensorFlow.js stubs that train a 16-factor matrix factorization (MF) model on MovieLens 100k. Implemented exactly as the docstring suggests, the L2 penalty is applied to the full factor matrices in every mini-batch: the factors collapse to about 1.8e-4, the model predicts the training mean (test RMSE 1.2289, equal to the baseline), and all 11 self-checks still pass. With a per-batch penalty and a restored best checkpoint, training on WebGL selects epoch 3 (validation RMSE 1.0208) and reaches test RMSE 1.0725, 12.7% below the mean-rating baseline. Under the same protocol, however, MF reaches Recall@10 = 0.0078, while recommending the most-rated unseen films gives 0.0845. Comparing two users shows why: the long-time user's Top-10 repeats his main genres through films with 1-8 training ratings, and the newcomer gets light comedies unrelated to his thrillers. MF spreads its lists over 739 distinct films against 77 for popularity, but reaches only 52 of 861 users with a hit against 283. Key takeaway: a lower rating error did not produce a more useful list, so the list itself has to be evaluated.

Index Terms - matrix factorization, TensorFlow.js, Recall@K, popularity baseline, AI-assisted debugging

1. Introduction

Problem. Implement initializeFactors, trainMF, predictRating, selectBestCheckpoint, recommendTopK and recallAtK in week5/script.js with TensorFlow.js; select a checkpoint by validation RMSE and compare test RMSE with a training-mean baseline; produce Top-10 lists for two users; report Recall@10 with its exclusions; and answer one business question (Section 4.1) with numbers from my own code.

Motivation. A streaming platform does not show predicted ratings; it shows a short ranked list. A model can predict ratings better than the average and still fill that list with films nobody has watched.

Concrete example. The first recommendation for User 1 is "Perfect Candidate, A (1996)" with score 5.1450, above the 1-5 rating scale; the film has 3 ratings in the training set (Fig. 6a).

Contributions.

- Audit of the starter, prompt and data (Table I), including a regularizer that passes every test while erasing the model (failure case, commit f457296) and a selected checkpoint that was never used.
- Fixed implementation (ee2a6e8, a9b55b2, 3dac310), a popularity baseline under the identical Recall@10 protocol, and a two-user analysis on the business question.

2. Related Work

MF with user and item factor vectors and an L2-regularized squared-error objective follows Koren et al. [1]; the data are MovieLens 100k [2]. The readme's citation of [2] (authors, title, journal, year) matches the publisher record; I added volume, issue, pages and DOI. Training uses Adam [3] in TensorFlow.js [4], [5]. Cremonesi et al. [6] showed that a lower RMSE does not guarantee better top-N accuracy and that a non-personalized popularity method can come close to sophisticated algorithms - the effect observed in Section 4. Starter and prompt: [7]; agent: [8].

Alternatives considered.

- **L2 on the full factor matrices every batch** (docstring suggestion): implemented first as the failure case and rejected - see Section 5.
- **Train-only candidate pool for recommendTopK** (readme 5.1): rejected for the contract, because self-checks 9-10 require a movie absent from training to be recommendable; the train-only variant is kept as recallAtKTrainOnly and gives the same Recall@10 to all printed digits (Fig. 3).
- **Clamping ranking scores to [1, 5]:** rejected - clamping would create ties at 5.0 and hide that the model over-scores rarely rated films; predictRating still clamps.

3. Method

Approach. Read-only audit by the agent -> challenge its claims against an independent re-computation -> implement the stubs exactly as suggested (failure case f457296) -> fix (ee2a6e8, a9b55b2) -> verify in Safari on WebGL -> fix two defects found in the browser (3dac310) -> analysis.

#	Location	Finding (evidence)	Resolution
1	trainMF docstring (L2 term)	Penalty on all (944 + 1,683) x 16 = 42,032 factor entries is added to the loss of each of the 1,245 mini-batches per epoch; factors fall to ~1.8e-4 and RMSE equals the baseline (val 1.1830, test 1.2289) while 11 / 11 checks pass	Penalty on the gathered batch rows divided by batch size (ee2a6e8)
2	trainMF check	Only asserts "one finite history row per epoch"; nothing compares the trained RMSE with the baseline	Baseline val/test shown next to the held model in the status line (a9b55b2)
3	runPipeline / trainMF	Best checkpoint selected but the last-epoch weights stay in the model; no test RMSE reported; history not reset between runs	Clone best weights, restore with assign(); held-model test RMSE in status; history reset (ee2a6e8)
4	initializeFactors / shuffle	Unseeded random init and shuffle: runs not reproducible	Seeded tf.randomNormal (42 / 43) and a seeded LCG shuffle; two browser runs gave identical curves
5	readme 5.1 vs checks 9-10	Train-only candidate rule and the harness are mutually exclusive	Contract kept; train-only variant reported (identical Recall@10)

#	Location	Finding (evidence)	Resolution
6	data.js titles	u.item is ISO-8859-1 but decoded as UTF-8; 9 titles corrupted	Re-decoded with TextDecoder("iso-8859-1") (a9b55b2)
7	u.item duplicates, id 267	Same title under two ids: 39 test ratings are twins of a film the user already rated; id 267 is "unknown" (7 / 2 / 0 ratings in train / val / test)	Twins of seen films and id 267 excluded from Top-10
8	readme 3 (shape)	Dense matrix is (943 + 1) x (1682 + 1); sparsity must use 943 x 1,682	Density 100,000 / 1,586,126 = 6.30%, sparsity 93.70%

Table I. Main findings. Withdrawn after re-check: the floor-based split counts (documented behaviour, not a defect).

Implementation. Prediction r = mu + p_u . q_i (mu = training mean), clamped to [1, 5] in predictRating. Each Adam step gathers the batch rows with tf.gather, computes squared error plus reg x (sum p_u^2 + sum q_i^2) / batch size, and calls optimizer.minimize with tf.train.adam (script.js:496-533; L2 line 533); factors start from tf.randomNormal(..., 0, 0.05, 'float32', 42 / 43) wrapped in tf.variable (431-432); the best epoch is kept with clone() and restored with assign() (551-558); recommendTopK scores all movies with itemFactors.matMul(userRow, false, true), masks seen films and takes tf.topk (648-667). The list was confirmed with grep on the committed file, excluding comment lines; every function is in the TF.js 4.22.0 API reference [5].

Tools. OpenCode 1.18.35 with "Big Pickle", then "Nemotron 3 Ultra Free" (OpenCode Zen) [8]; TensorFlow.js 4.22.0 (index.html:124); Safari on macOS, backend **webgl**; Node.js v24.10.0. **Configuration.** 16 factors, Adam lr 0.01, reg 0.05, batch 64, 20 epochs; split 79,619 / 9,596 / 10,785 ratings (0 users skipped).

4. Experiments

Setup. MovieLens 100k: 943 users, 1,682 movies, 100,000 ratings, split per user by timestamp 80 / 10 / 10. Relevant = test rating >= 4 on a movie present in training. All numbers come from the page or the Node harness at the stated commit.

Model	Val RMSE	Test RMSE	Recall@10	Users with hit / distinct films
Training-mean baseline	1.1830	1.2289	-	-
MF, full-matrix L2 (f457296)	1.1830	1.2289	-	factors ~1.8e-4; 11 / 11 pass
MF, per-batch L2, epoch 3 restored	**1.0208**	**1.0725**	**0.0078**	52 / 861; **739** films
MF, last epoch (20) if not restored	1.0436	not reported	-	train 0.7070
Most-rated unseen films	-	-	**0.0845**	283 / 861; **77** films

Table II. Results on WebGL (MF rows; users with hit and distinct films from the console, Fig. 3b) and from the page (popularity). Exclusions for both Recall rows: 13 unseen relevant ids, 82 users without a relevant candidate; 861 users evaluated. Distinct films = distinct ids across the Top-10 lists of all 943 users.

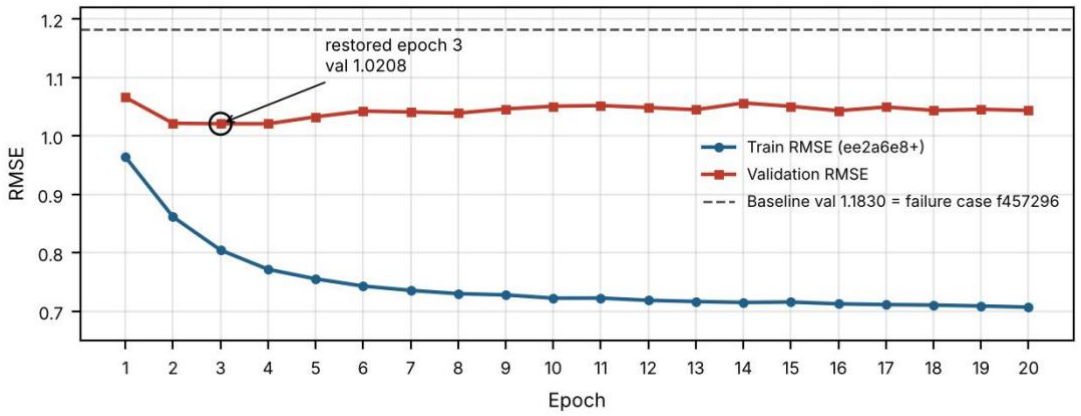


Fig. 1. Training curve: the 20 train / validation RMSE values from the page's Training curve table (identical in all three runs), re-plotted with matplotlib by Claude (Section 6). Validation RMSE is lowest at epochs 3-4 (1.0208, earliest kept) and rises to 1.0436, while train RMSE keeps falling to 0.7070.

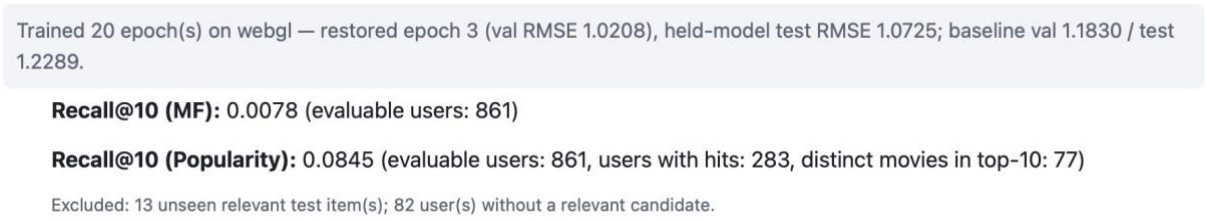


Fig. 2. Status line and Recall panel after training (commit 3dac310).

Arithmetic. Test gain over the baseline: 1.2289 - 1.0725 = 0.1564, i.e. 0.1564 / 1.2289 = 12.7% (validation: 0.1622 / 1.1830 = 13.7%). Restoring epoch 3 instead of keeping epoch 20 saves 1.0436 - 1.0208 = 0.0228 validation RMSE. Popularity vs MF Recall@10: 0.0845 / 0.0078 = 10.8 times; users with a hit 283 / 861 = 32.9% vs 52 / 861 = 6.0%; distinct films 739 / 77 = 9.6 times more for MF. Spot check of one Recall row: user 303 has 12 relevant films and 1 hit, 1 / 12 = 0.0833 as shown.

Hand computation. Fargo (id 100) for User 1: I exported mu and the two 16-value factor rows from the console and computed mu + SUMPRODUCT(p_1, q_100) in Excel: 3.58060262 + 1.13666436 = 4.717266975, which rounds to the page score 4.7173 (Fig. 4).

Verification. (i) Self-checks: 4 / 0 / 7 in the starter, 11 / 0 / 0 for the failure case and after every fix (Fig. 5). (ii) Three 20-epoch browser runs (before and after 3dac310) gave identical curves, status lines and Top-10 lists. (iii) recallAtK and recallAtKTrainOnly both return 0.007787810613693653 (Fig. 3). (iv) All 20 recommended films for Users 1 and 36 are absent from their training and validation ratings, and the popularity Recall, user histories and exclusions match an independent re-computation (Section 6).



(a) contract pool vs train-only pool



(b) User 1 row, users with a hit, distinct films

Fig. 3. Safari console after training.

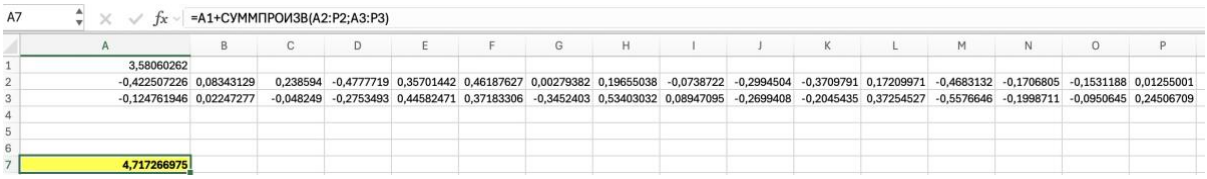


Fig. 4. Hand computation in Excel: mu (A1), p_1 (row 2), q_100 (row 3); A7 = A1 + SUMPRODUCT(A2:P2; A3:P3) = 4.717266975.

```
HW5 self-checks — pass 11, fail 0, pending 0 (of 11)

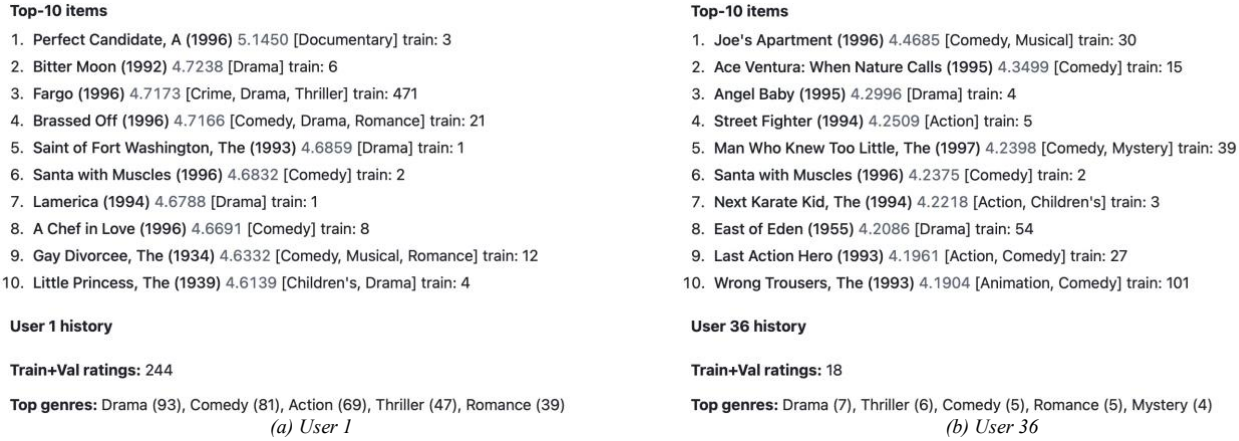
PASS    computeBaselineRMSE: toy split matches the hand-computed value — rmse=1.0000
PASS    computeBaselineRMSE: empty split returns 0 (no division by zero) — guarded
PASS    fmt: formats numbers and guards non-finite input — ok
PASS    movieTitle: resolves ids and falls back for unknown ids — ok
PASS    predictRating: clamps to [1, 5] and falls back to the global mean — clamped=5, fallback=3
PASS    initializeFactors: allocates tf.Variable factors and the global mean — user [944,4], item
[4,4]
PASS    trainMF: returns one finite history row per epoch — e1:1.1611 e2:1.1515 e3:1.1363
PASS    selectBestCheckpoint: smallest valRMSE wins, ties keep the earliest epoch — best epoch 1
PASS    recommendTopK: excludes train+val items but allows test-rated items — 3, 4, 5
PASS    recallAtK: relevant movie at rank 1 scores 1.0; at rank 11 scores 0.0 — rank1=1.0000
rank11=0.0000
PASS    recallAtK: empty test set returns 0 without dividing by zero — guarded
```

Fig. 5. Self-checks in Safari after 3dac310: 11 pass / 0 fail / 0 pending.

4.1 Familiar choices vs. new discoveries

	User 1	**User 36**
History (train + val)	244 ratings; Drama 93, Comedy 81, Action 69, Thriller 47, Romance 39; 67 five-star ratings	18 ratings; Drama 7, Thriller 6, Comedy 5, Romance 5, Mystery 4; five-star: Air Bud, Mad City, Spawn, That Darn Cat!, Washington Square, Telling Lies in America
Top-10 genres	Drama or Comedy in 9 / 10; Action 0	Comedy 6, Action 3, Drama 2; Thriller 0
Training ratings of the 10 films	1, 1, 2, 3, 4, 6, 8, 12, 21, 471 (median 5)	2, 3, 4, 5, 15, 27, 30, 39, 54, 101 (median 21)
Scores	4.6139 - 5.1450	4.1904 - 4.4685
Test hits	Fargo (5 in test): 1 of 19 relevant	0 of 2 relevant

Table III. Two users compared (history panel after 3dac310, Top-10 lists in Fig. 6).



(a) User 1 *(b) User 36*

Fig. 6. Top-10 lists with genres and training counts, and the history panel (commit 3dac310).

Answer. The lists mostly repeat genres and add little validated discovery. **User 1** watches broadly but mostly Drama and Comedy; 9 of his 10 recommendations carry one of these genres and none is Action, although Action is his third genre (69 ratings). The titles look like discoveries, but 7 of 10 have at most 8 training ratings (median 5), and the top score 5.1450 lies above the rating scale. With 16 free values per film and 1-3 ratings, the model most likely fits those few ratings rather than a stable taste signal, so these picks look novel but are weakly supported; the one list item he actually rated highly later (5 in test) is the mainstream Fargo with 471 training ratings. **User 36** rated 18 films, mostly recent ones (9 of the 10 highest-rated films in his history panel are 1997 releases), with Drama and Thriller first. His list shifts him to light Comedy and Action from 1993-1996 (Joe's Apartment, Ace Ventura, Street Fighter) with lower scores, and contains no Thriller: with 18 ratings his user vector is weak and the list is closer to a generic one (median 21 training ratings). Neither of his 2 relevant test films is found.

Value for a platform. Ranked by what users later rate highly, recommending the most-rated unseen films is 10.8 times better (Recall@10 0.0845 vs 0.0078) and gives at least one hit to 283 of 861 users (MF: 52), but all 943 lists draw on only 77 distinct films - safe, familiar and nearly the same for everyone. MF spreads its lists over 739 distinct films, so it does surface the catalogue, yet its "new" items are mostly unproven long-tail titles, which risks a viewer's trust when the first row starts with a film almost nobody has

rated. Decision on a reach axis: use popularity (or MF re-ranked within films with enough ratings) for the main row that must convert, and give MF a separate, labelled "discover" row; test both online on watch-through before replacing the default.

5. Discussion

5.1 Failure case

- Commit f457296 (stubs exactly as suggested): 11 / 11 checks pass, yet train / val / test RMSE are 1.1052 / 1.1830 / 1.2289 - the validation and test values equal the baseline; the factors shrink to about 1.8e-4, so every prediction is mu.

5.2 Root cause

- The penalty on all 42,032 factor entries enters each of the 1,245 batch losses per epoch, while each rating's error enters once per epoch and is averaged over 64; shrinkage dominates the data term. The only training check tests that RMSE values are finite.

5.3 Fix and verification

- Penalty on the gathered rows divided by batch size, plus checkpoint restore: validation 1.0208, test 1.0725 (Table II), reproduced in three browser runs; checks still 11 / 0 / 0.

5.4 What worked / what surprised me

- Running a deliberately "as suggested" version first made the trap visible: the tests could not tell a dead model from a working one.
- The RMSE gain did not carry over to ranking: the simplest baseline beat MF by an order of magnitude at Top-10.

5.5 AI errors I caught

- Audit: sparsity computed on the padded 944 x 1,683 matrix; movie 267 listed as never in training (it has 7 / 2 / 0); the candidate-pool conflict understated; the documented floor rule called a defect.
- Commit a9b55b2 showed "Train+Val ratings: 0" for every user (select value as a string compared with === to a number) and omitted the promised exclusion counts; both fixed in 3dac310 after the browser check.

5.6 Next improvement

- Add item and user bias terms and a minimum-support filter for candidates (e.g. >= 20 training ratings), then report Recall@10 together with catalogue coverage, so that accuracy and discovery are measured on the same lists.

6. AI Usage Disclosure

- **OpenCode** 1.18.35, models "Big Pickle" and, after its free limit was reached, "Nemotron 3 Ultra Free" (OpenCode Zen), in one session ("Week5 starter code and prompt audit", ses_ecedc7853ffeJUpInlIpWN4nnV): audit, implementation, fixes, Node harness; commits f457296, ee2a6e8, a9b55b2, 3dac310. TLS certificate errors were resumed with "continue" in the same session. Exported unmodified with opencode export as zaporozhets_a05_session.json (1.5 MB).
- **Claude Opus 5.5** (claude.ai chat): step-by-step guidance and the prompts sent to OpenCode; an independent Python re-computation of the split, exclusions, user histories and the popularity baseline, written before the agent implemented it (all values matched); reference checks; Fig. 1 re-plotted from the page's table; the skeleton of this report. Not part of session.json. Three of its statements were wrong and corrected: a Safari console command using await; an initial claim that all 18 of User 36's films were 1997 releases (15 are, by its re-computation); and a draft of this report naming tf.matMul, which my grep showed is called as a tensor method (.matMul, script.js:652).

Personally verified: the status line, training curve, Top-10 lists with histories, Recall panel, self-checks and console results in Safari (Figs. 1-3, 5, 6); the Fargo prediction in Excel (Fig. 4); the arithmetic in Section 4; the TF.js calls in the committed code (grep); the commit history (git log).

Trusted without verification: the failure-case factor magnitude (~1.8e-4) and train RMSE 1.1052 reported by the agent; the 9 corrupted titles and 39 twin ratings (agent and second AI, not re-counted by me); the slight WebGL vs CPU difference with the same seeds (observed by the agent, not quantified).

References

[1] Y. Koren, R. Bell, and C. Volinsky, "Matrix factorization techniques for recommender systems," Computer, vol. 42, no. 8, pp. 30-37, 2009. [Online]. Available: https://doi.org/10.1109/MC.2009.263. [Accessed: 2026-10-13].

[2] F. M. Harper and J. A. Konstan, "The MovieLens datasets: History and context," ACM Trans. Interact. Intell. Syst., vol. 5, no. 4, pp. 1-19, 2015. [Online]. Available: https://doi.org/10.1145/2827872. [Accessed: 2026-10-13].

[3] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in Proc. 3rd Int. Conf. Learning Representations (ICLR), 2015. [Online]. Available: https://arxiv.org/abs/1412.6980. [Accessed: 2026-10-13].

[4] D. Smilkov et al., "TensorFlow.js: Machine learning for the web and beyond," in Proc. Machine Learning and Systems (MLSys), vol. 1, 2019, pp. 309-321. [Online]. Available: https://arxiv.org/abs/1901.05350. [Accessed: 2026-10-13].

[5] Google, "TensorFlow.js API," version 4.22.0. [Online]. Available: https://js.tensorflow.org/api/latest/. [Accessed: 2026-10-13].

[6] P. Cremonesi, Y. Koren, and R. Turrin, "Performance of recommender algorithms on top-N recommendation tasks," in Proc. 4th ACM Conf. Recommender Systems (RecSys), 2010, pp. 39-46. [Online]. Available: https://doi.org/10.1145/1864708.1864721. [Accessed: 2026-10-13].

[7] dryjins, "RecSys-LLMs, week5," GitHub repository, 2026. [Online]. Available: https://github.com/dryjins/RecSys-LLMs/tree/main/week5. [Accessed: 2026-10-13].

[8] Anomaly, "CLI," OpenCode Documentation, 2026. [Online]. Available: https://opencode.ai/docs/cli/. [Accessed: 2026-10-13].